3c:

(c) Evaluate the base_model on the test set (X_test) using classification_report and confusion_matrix from the sklearn.metrics library. Report these numbers in your .pdf writeup file using screenshots.

```
[5]: ## INSERT YOUR CODE HERE ##
# Save the original Keras model to HDF5 file
base_model.save('original_model.h5')
```

```
[5]: # Save the original Keras model to HDF5 file
base_model.save('original_model.h5')
y_pred = base_model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

print(classification_report(y_test, y_pred_classes))
print(confusion_matrix(y_test, y_pred_classes))
```

```
788/788 [==============================] - 0s 324us/step
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     11742
           1       1.00      1.00      1.00     13453

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195

[[11721    21]
 [   18 13435]]
```

4b:

(b) Evaluate the tflite_quant_model on the test set (X_test) using classification_report and confusion_matrix from the sklearn.metrics library. Report these numbers in your .pdf writeup file using screenshots.

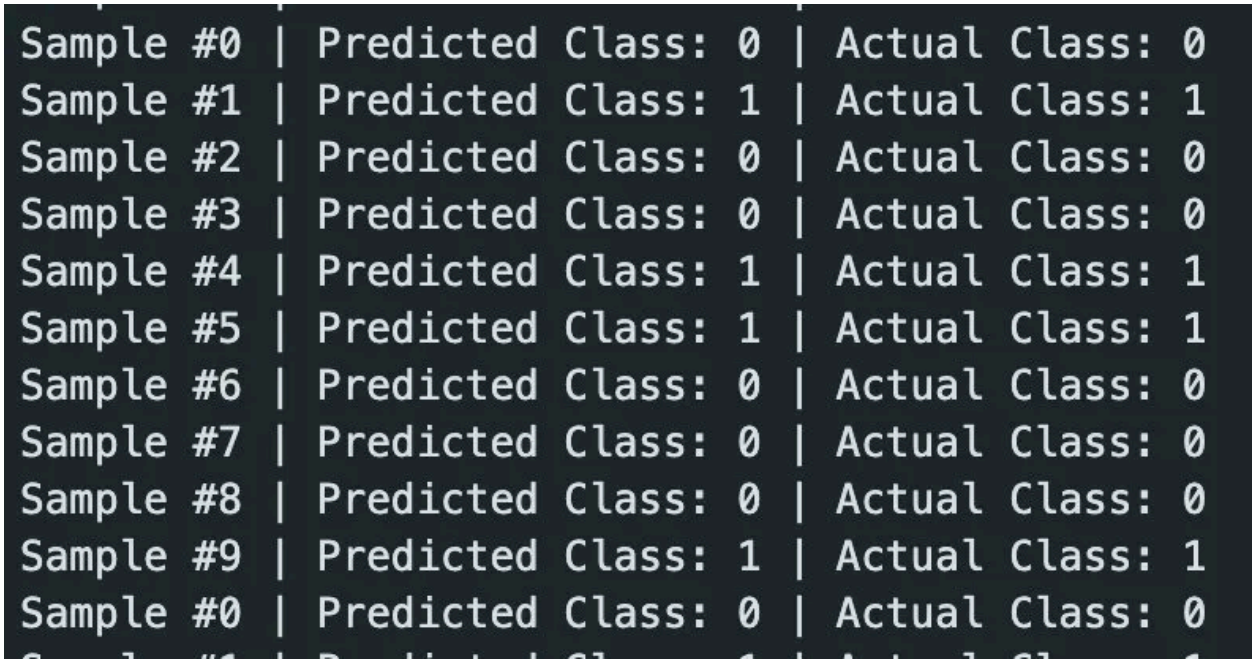
```
[ ]: ## INSERT YOUR CODE HERE ##
interpreter = tf.lite.Interpreter(model_content=tflite_quant_model)
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

quant_preds = []
for i in range(len(X_test)):
    sample = X_test[i:i+1].astype(np.float32)
    interpreter.set_tensor(input_details[0]['index'], sample)
    interpreter.invoke()
    out = interpreter.get_tensor(output_details[0]['index'])
    quant_preds.append(np.argmax(out))
quant_preds = np.array(quant_preds)

print(classification_report(y_test, quant_preds))
print(confusion_matrix(y_test, quant_preds))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     11742
           1       1.00      1.00      1.00     13453

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195

[[11721    21]
 [   21 13432]]
```

5c:

```
Setup started
Sample #0 | Predicted Class: 1 | Actual Class: 1
Sample #1 | Predicted Class: 1 | Actual Class: 1
Sample #2 | Predicted Class: 0 | Actual Class: 0
Sample #3 | Predicted Class: 0 | Actual Class: 0
Sample #4 | Predicted Class: 1 | Actual Class: 1
```

5d:



Notes:
TensorFlow Lite Micro doesn't support hybrid (dynamic-range-quantized) models. Its FullyConnected kernel rejects these models during AllocateTensors, saying, "Hybrid models are not supported on TFLite Micro." This happens because dynamic range quantization uses float activations with int8 weights, so the tflite_quant_model can't be allocated on the Nano. For on-device deployment in Question 5, we had to use a plain float TFLite model. It's got the same architecture and weights but with float I/O. We converted this and generated network_model.h, which runs just fine with TFLM. The dynamic range quantization work from Question 4 remains unchanged in the notebook.